

DSA STUDY BLUEPRINT SERIES

73. Circular Queue in Data Structure

Recycling Queue memory spaces using Modulo Indices

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Circular Queue:** Avoids space wastage by wrapping pointers: $\text{rear} = (\text{rear} + 1) \% \text{size}$.
- Maintains bounds tracking using elements count variables.

Memory & Execution Blueprint

[Tail] wrapping around: $(\text{tail} + 1) \% \text{Capacity} \Rightarrow$ uses vacated index 0.

C++ Implementation Reference

Standard code layout and syntax

```
class MyCircularQueue {  
    vector q;  
    int head = 0, tail = -1, sz, capacity;  
public:  
    MyCircularQueue(int k) : capacity(k), sz(0), q(k) {}  
    bool enQueue(int value) {  
        if (sz == capacity) return false;  
        tail = (tail + 1) % capacity;  
        q[tail] = value; sz++; return true;  
    }  
};
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
enQueue / deQueue	$O(1)$ time, $O(K)$ space

Key Practices & Gotchas

- **Important:** Properly handle empty vs full queue states using size counters.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace circular queue pointer wrapping when capacity = 3:

Operation	Queue data layout	Pointers [front, rear]	Size count
enqueue(10)	[10, _, _]	[0, 0]	1
enqueue(20)	[10, 20, _]	[0, 1]	2
dequeue()	[_ , 20, _]	[1, 1]	1 (wraps correctly)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Wrap-around indices:** Modulo indexing updates circular buffer limits dynamically.
- **Deque mechanics:** Doubly ended queues allow fast $O(1)$ front and rear pushes and pops.
- **Related LeetCode Problems:** #239 Sliding Window Maximum, #622 Design Circular Queue, #225 Implement Stack using Queues.